

Guide 3 — Sécurité et administration

Ta mission : que le jury ne trouve aucune faille, et que l'espace admin (Filament) couvre toutes les fonctionnalités « administration ». Tu es aussi l'adjoint du chef d'équipe : tu tries et tu déploies quand il dort.

1. Ce que le jury va tester, et où c'est géré chez nous

Famille testée	Protection en place	Où
Authentification	Fortify : mots de passe hachés, 2FA, confirmation de mot de passe	starter kit
Brute force	Connexion bloquée après 5 essais (429)	Fortify + test SecurityTest
Contrôle d'accès	Policies + <code>\$this->authorize()</code> dans chaque action	app/Policies/ , pages
Rôles	role (user/admin), hors <code>#[Fillable]</code> , admin Filament réservé	User , EditUser
Validation des entrées	<code>rules()</code> dans chaque formulaire	pages >form
Endpoints protégés	Toutes les routes métier dans le groupe <code>auth</code>	routes/features.php
Fuite de données	<code>APP_DEBUG=false</code> , secrets masqués, <code>#[Hidden]</code> sur User	.env , User
En-têtes HTTP	X-Frame-Options, nosniff, Referrer-Policy, HSTS	SecurityHeaders

2. Les 5 attaques qu'un juré fera en 10 minutes

- Changer l'ID dans l'URL** : `/signalements/12/edit` avec le compte d'un autre → doit donner **403**.
- Appeler une action Livewire à la main** (outils du navigateur) : `delete(12)` sur un élément qui n'est pas à lui → **403**.
- Ajouter un champ au formulaire** : `role=admin` , `user_id=1` , `statut=valide` → **ignoré**.
- Accéder à `/admin`** avec un compte normal → **403**.
- Injecter du HTML/JS** dans un champ texte (`<script>alert(1)</script>`) → affiché comme du texte, jamais exécuté.

Chaque fonctionnalité livrée doit résister à ces 5 tests. **Fais-les à la main avant chaque déploiement important.**

3. Les règles dans le code

Chaque méthode publique d'un composant vérifie les droits :

```
public function delete(int $id): void
{
    $signalement = Signalement::findOrFail($id);
    $this->authorize('delete', $signalement); // ← sans cette ligne : faille
    $signalement->delete();
}
```

Propriétés sensibles verrouillées (le navigateur ne peut pas les modifier) :

```
#[Locked]
public ?Signalement $record = null;
```

Champs sensibles hors `#[Fillable]` , assignés dans le code :

```
$signalement = new Signalement($validated);
$signalement->user()->associate(auth()->user()); // jamais depuis le formulaire
$signalement->statut = 'en_attente';
$signalement->save();
```

Uploads :

```
'photo' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
// stockage avec un nom aléatoire :
$chemin = $this->photo->store('signalements', 'public');
```

Limiter une action sensible (signalement, message, appel IA...) :

```
$cle = 'signalement:'.auth()->id();
if (RateLimiter::tooManyAttempts($cle, 10)) {
    $this->addError('titre', 'Trop de signalements. Réessayez dans une minute.');
```

```
    return;
}
RateLimiter::hit($cle, 60);
```

Données personnelles : ne jamais afficher l'e-mail ou le téléphone d'un autre utilisateur sauf si le sujet l'exige ; dans ce cas, le faire passer par une Policy (`viewContact`).

4. Fonctionnalités « sécurité » annoncées : réponses toutes prêtes

Annonce probable	Ce qu'on a / ce qu'on fait
« Authentification sécurisée »	Déjà là (hash, 2FA, limite d'essais) → à déclarer au jury
« Rôles utilisateur / administrateur »	Déjà là → déclarer, montrer <code>/admin/users</code>
« Restreindre certaines actions aux connectés »	Déjà là (groupe <code>auth</code>) → déclarer
« Limiter les tentatives de connexion »	Déjà là → déclarer, test <code>SecurityTest</code>
« Protéger une zone sensible / endpoint »	Policy dédiée + test 403 (15 min)
« Validation renforcée »	Compléter <code>rules()</code> (tailles, formats, <code>Rule::in</code>), messages en français
« Journal des actions »	Voir section 6 (30 min)
« Contrôle d'accès aux données sensibles »	Policy <code>view</code> restrictive + masquage dans les vues

Plusieurs fonctionnalités de sécurité sont donc des **points gratuits**, à condition de les déclarer et de savoir les démontrer.

5. Tests de sécurité (modèle à copier)

```
test('un autre utilisateur ne peut pas modifier', function () {
    $signalement = Signalement::factory()->create();

    $this->actingAs(User::factory()->create())
        ->get(route('signalements.edit', $signalement))
        ->assertForbidden();
});

test('le formulaire refuse un niveau inventé', function () {
    Livewire::actingAs(User::factory()->create())
        ->test('pages::signalements.form')
        ->set('titre', 'Test')
        ->set('niveau', 'apocalypse')
        ->call('save')
        ->assertHasErrors(['niveau']);
});

test('on ne peut pas se donner un statut réservé', function () {
    Livewire::actingAs($user = User::factory()->create())
        ->test('pages::signalements.form')
        ->set('titre', 'Test')
        ->call('save');

    expect(Signalement::where('user_id', $user->id)->first()->statut)->toBe('en_attente');
});
```

6. Journal des actions (fonctionnalité fréquente)

```
php artisan make:model ActionLog -m
```

```
// migration
$table->id();
$table->foreignId('user_id')->nullable()->constrained()->onDelete('cascade');
$table->string('action', 50); // created, updated, deleted, login...
$table->string('subject_type')->nullable();
$table->unsignedBigInteger('subject_id')->nullable();
$table->string('ip', 45)->nullable();
$table->timestamps();
```

```
// app/Models/ActionLog.php
#[Fillable(['user_id', 'action', 'subject_type', 'subject_id', 'ip'])]
class ActionLog extends Model
{
    public static function record(string $action, ?Model $subject = null): void
    {
        static::create([
            'user_id' => auth()->id(),
            'action' => $action,
            'subject_type' => $subject ? class_basename($subject) : null,
            'subject_id' => $subject?->getKey(),
            'ip' => request()->ip(),
        ]);
    }
}
```

Appel : `ActionLog::record('deleted', $signalement);` dans les actions importantes. Affichage : ressource Filament en lecture seule.

7. Filament : l'administration en quelques minutes

```
php artisan make:filament-resource Signalement --generate
```

Crée liste, création, modification dans `/admin`. Puis on ajuste :

Formulaire (`Schemas/SignalementForm.php`) : champs lisibles, `Select` pour les enums.

```
Select::make('niveau')->options(array_combine(Signalement::NIVEAU_OPTIONS,
Signalement::NIVEAU_OPTIONS))->required(),
```

Tableau (`Tables/SignalementsTable.php`) : colonnes, badges, filtres, action de modération.

```

use Filament\Actions\Action;
use Filament\Tables\Columns\TextColumn;
use Filament\Tables\Filters\SelectFilter;

TextColumn::make('niveau')->badge()->color(fn (string $state) => match ($state) {
    'critique' => 'danger', 'moyen' => 'warning', default => 'gray',
}),

SelectFilter::make('niveau')->options(array_combine(Signalement::NIVEAU_OPTIONS,
Signalement::NIVEAU_OPTIONS)),

Action::make('resoudre')
    ->label('Marquer résolu')
    ->icon('heroicon-o-check')
    ->requiresConfirmation()
    ->visible(fn (Signalement $record) => $record->statut !== 'resolu')
    ->action(function (Signalement $record) {
        $record->statut = 'resolu';    // statut hors Fillable : assignation explicite
        $record->save();
    }),

```

Statistiques (tableau de bord admin) :

```
php artisan make:filament-widget StatsOverview --stats-overview
```

```

use Filament\Widgets\StatsOverviewWidget\Stat;

protected function getStats(): array
{
    return [
        Stat::make('Signalements', Signalement::count()),
        Stat::make('Critiques', Signalement::where('niveau', 'critique')->count())-
        >color('danger'),
        Stat::make('Utilisateurs', User::count()),
    ];
}

```

Rappels :

- Tout ce qui est dans Filament est déjà réservé aux admins, mais la Policy s'applique aussi.
- Ne jamais afficher de mot de passe, de secret 2FA ou de jeton dans une ressource.
- Champ hors `#[Fillable]` modifié depuis Filament : l'assigner explicitement (voir `EditUser::handleRecordUpdate`), sinon il est ignoré sans erreur.

8. Avant chaque livraison : `/audit-secu`

Dans Claude Code : `/audit-secu` (toute l'app) ou `/audit-secu signalements`. Il vérifie routes, autorisations, assignation de masse, validation, XSS, fuites, limitations. Corrige les points

critiques immédiatement, note les autres.

Vérifications en production :

```
grep -E '^(APP_ENV|APP_DEBUG)= ' ~/webcup-2026/.env      # production / false
curl -sI https://virtualvisionsy.madagascar.webcup.hodi.cloud | grep -iE 'x-frame|x-
content|strict-transport'
```

9. Rôle d'adjoint (pendant que le chef dort)

- Tu tries les annonces (`/triage`), tu mets à jour le board.
- Tu merges et tu déploies (`bash ~/webcup-2026/deploy.sh`), puis tu testes en ligne.
- En cas de problème en production : suivre la section « Quand ça casse » du guide du chef d'équipe.